

**САНКТ-ПЕТЕРБУРГСКИЙ НАЦИОНАЛЬНЫЙ  
ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**Дисциплина:** Фронт-энд разработка

Отчет

Лабораторная работа №3

Выполнила:

Рудникова Виктория

К3343

Проверил:

Добряков Д. И.

Санкт-Петербург

2026 г.

## Задача

Мигрировать ранее разработанное приложение (в рамках ЛР1 и ЛР2) на фреймворк Vue.JS.

Каким требованиям ваше приложение должно соответствовать:

- Должен быть подключён роутер
- Должна быть реализована работа с внешним API (желательно посредством axios)
- Разумное деление на компоненты (продемонстрируйте понимание компонентного подхода)
- Использование composables для выделения повторяющегося функционала в отдельные файлы

## Ход работы

### 1. Инициализация проекта

Начинаем с инициализации Vue-проекта:

```
○ vorudnikova@MSK-T7GWG604V lab3 % npm init vue@latest  
  
  > npx  
  > create-vue  
  
  Vue.js – The Progressive JavaScript Framework  
  ◆ Project name (target directory):  
    rent-vue
```

### 2. Реализуем роутер

В новом проекте первым делом создадим роутер – в файле `src/router/index.js` пропишем роуты до регистрации, входа, личного кабинета, домашней страницы (/):

```

7   const router = createRouter({
8     history: createWebHistory(import.meta.env.BASE_URL),
9     routes: [
10      {
11        path: '/',
12        name: 'home',
13        component: HomeView,
14        meta: { title: 'Главная' },
15      },
16      {
17        path: '/login',
18        name: 'login',
19        component: LoginView,
20        meta: { title: 'Вход' },
21      },
22      {
23        path: '/register',
24        name: 'register',
25        component: RegisterView,
26        meta: { title: 'Регистрация' },
27      },
28      {
29        path: '/profile',
30        name: 'profile',
31        component: ProfileView,
32        meta: { title: 'Личный кабинет' },
33      },
34    ],
35  })
36

```

И отредактируем App.vue, чтобы всё было корректно:

```

1  <script setup>
2  </script>
3
4  <template>
5    <router-view />
6  </template>
7
8  <style scoped>
9  </style>
10

```

Также добавим четыре представления, где предполагаются будущие страницы (пока вставим туда заглушки):

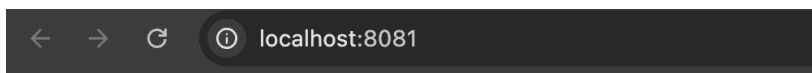
```

  views
  ▼ HomeView.vue      U
  ▼ LoginView.vue     U
  ▼ ProfileView.vue   U
  ▼ RegisterView.vue  U

```

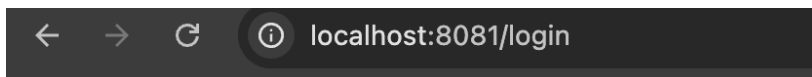
```
lab3 > K3343 > Рудникова Виктория > lab3 > rent-vue > src > views > HomeView.vue
1  <template>
2    <div>
3      <h1>Поиск объявлений</h1>
4      <p>Главная страница RentEasy – здесь будет поиск недвижимости.</p>
5    </div>
6  </template>
7
8  <script setup>
9    defineOptions({ name: 'HomeView' })
10 </script>
```

Результат:



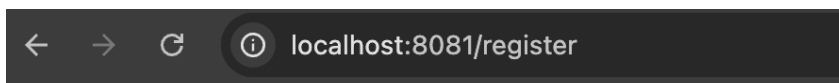
## Поиск объявлений

Главная страница RentEasy - здесь будет поиск недвижимости.



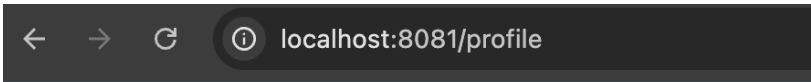
## Вход

Страница входа - форма будет добавлена позже.



## Регистрация

Страница регистрации - форма будет добавлена позже.



## Личный кабинет

Профиль пользователя - контент будет добавлен позже.

Роутер работает корректно.

### 3. Базовый layout

Установим Bootstrap и импортируем в **main.js**:

```
1 import { createApp } from 'vue'
2 import { createPinia } from 'pinia'
3
4 import 'bootstrap/dist/css/bootstrap.min.css'
5 import 'bootstrap'
6 import './assets/main.css'
7
```

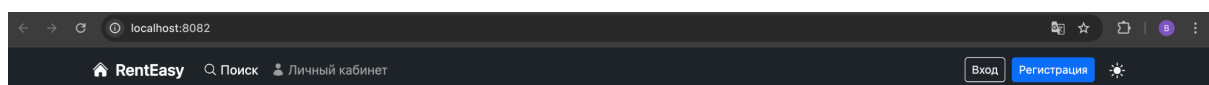
Добавим спрайт из иконок (**public/icons.svg**) и используем его в **index.html**:

```
<body>  
<svg xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink" aria-hidden=""  
  <defs>  
    <symbol id="icon-sun" viewBox="0 0 24 24" fill="currentColor">  
      <path d="M12 2.25a7.5 7.5 0 1.75v2.25a7.5 7.5 0 1-1.5 0V3a7.5 7.5 0 1.75-.75zM7.5  
    </symbol>  
    <symbol id="icon-moon" viewBox="0 0 24 24" fill="currentColor">  
      <path fill-rule="evenodd" d="M9.528 1.718a7.5 7.5 0 1.162.819A8.97 8.97 0 009 6a9 9 0 011.718 9.528a7.5 7.5 0 1-1.718 1.718Z"/>  
    </symbol>  
    <symbol id="icon-home" viewBox="0 0 24 24" fill="currentColor">  
      <path d="M11.47 3.841a7.5 7.5 0 11.06 0l8.69 8.69a7.5 7.5 0 101.06-1.061L8.689-8.689a7.5 7.5 0 1-1.061 1.061Z"/>  
      <path d="M12 5.432l18 18.159 8.159c.03.03.06.058.091.086v6.198c0 1.035-.84 1.875-1.875 1.875Z"/>  
    </symbol>  
    <symbol id="icon-search" viewBox="0 0 24 24" fill="currentColor">  
      <path fill-rule="evenodd" d="M10.5 3.75a6.75 6.75 0 100 13.5 6.75 6.75 0 100-13.5ZM20.25 7.5a6.75 6.75 0 110 13.5 6.75 6.75 0 110-13.5Z"/>  
    </symbol>  
    <symbol id="icon-user" viewBox="0 0 24 24" fill="currentColor">  
      <path fill-rule="evenodd" d="M7.5 6a4.5 4.5 0 119 0 4.5 4.5 0 11-9 0z M3.75 17.25 20.25 17.25a4.5 4.5 0 110 9 4.5 4.5 0 110-9Z"/>  
    </symbol>  
  </defs>  
</svg>  
<div id="app"></div>  
<script type="module" src="/src/main.js"></script>  
</body>
```

Создадим файл **layout/BaseLayout.vue**: опишем навбар и контент, также добавим ссылки на иконки. Подключим BaseLayout в **App.vue**:

```
1  <script setup>
2  | import BaseLayout from '@layout/BaseLayout.vue'
3  </script>
4
5  <template>
6  |   <BaseLayout />
7  </template>
8
9  <style scoped>
10 </style>
```

Результат:



## Поиск объявлений

Главная страница RentEasy - здесь будет поиск недвижимости.



## Личный кабинет

Профиль пользователя - контент будет добавлен позже.



## Вход

Страница входа - форма будет добавлена позже.



## Регистрация

Страница регистрации - форма будет добавлена позже.

Базовый лейаут и иконки успешно применены.

## 4. API

Установим `axios` и создадим папку `src/api`, в которую положим файлы: **`index.js`**, **`instance.js`**, **`users.js`** (методы для пользователей), **`listings.js`** (методы для объявлений).

Методы пользователей:

```
1  import instance from './instance.js'
2
3
4  class UsersApi {
5    constructor(axiosInstance) {
6      this.api = axiosInstance
7    }
8
9    /** Найти по email */
10   getByEmail(email) {
11     return this.api.get('/users', {
12       params: { email },
13     })
14   }
15
16   /** Создать пользователя */
17   create(userData) {
18     return this.api.post('/users', userData, {
19       headers: { 'Content-Type': 'application/json' },
20     })
21   }
22
23   /** Получить пользователя по id */
24   getById(id) {
25     return this.api.get(`/users/${id}`)
26   }
27 }
28
29 export default new UsersApi(instance)
```

Методы объявлений:

```
1  import instance from './instance.js'
2
3
4  class ListingsApi {
5    constructor(axiosInstance) {
6      this.api = axiosInstance
7    }
8
9    /** Получить все объявления */
10   getAll(params = {}) {
11     return this.api.get('/listings', { params })
12   }
13
14   /** Получить объявление по id */
15   getById(id) {
16     return this.api.get(`/listings/${id}`)
17   }
18 }
19
20 export default new ListingsApi(instance)
```

Также добавляем db.json со всеми нашими данными (пользователи и объявления):

```
1 {
2   "users": [
3     {
4       "id": 1,
5       "name": "Тест Тестов",
6       "email": "testovtest@gmail.com",
7       "phone": "+7 (999) 123-45-67",
8       "password": "123456"
9     },
10    {
11      "id": 2,
12      "name": "Снежана Абдурахманова",
13      "email": "snezhana_hi@gmail.com",
14      "phone": "+7 (981) 801-08-89",
15      "password": "qwerty"
16    },
17    {
18      "id": 3,
19      "name": "Максим Ашкудишкин",
20      "email": "ashk228@gmail.com",
21      "phone": "+7 (987) 641-31-51",
22      "password": "password"
23    }
24  ],
25  "listings": [
26    {
27      "id": 1,
28      "title": "Квартира-студия у метро Владимирская",
29      "address": "Санкт-Петербург, Центральный район",
30      "price": 45000,
31      "image": "studiya_v_centre.jpeg",
32      "rooms": 1,
33      "area": 24,
34      "floor": "4/6",
35      "type": "квартира"
36    }
37  ]
38 }
```

Не забываем добавить axios в зависимости и json-server в **package.json**:

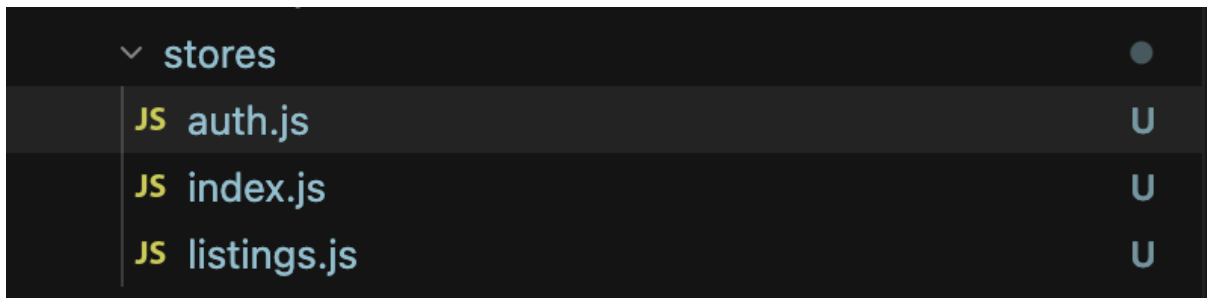
```
6   "scripts": {
7     "start": "vite --port 8080",
8     "dev": "vite",
9     "api": "npx json-server --watch db.json --port 3000",
10    "build": "vite build",
11    "preview": "vite preview",
12    "lint": "eslint . --ext .vue,.js,.jsx,.cjs,.mjs --fix --ignore-path .gitignore"
13  },
14  "dependencies": {
15    "axios": "^1.13.5",
16    "bootstrap": "^5.3.3",
17    "pinia": "^3.0.4",
18    "vue": "^3.5.27",
19    "vue-router": "^5.0.1"
20  },
```

Добавим хранилища:

- **auth.js** реализует методы регистрации и входа/выхода



- **listings.js** реализует методы получения объявлений: всех или по id
- **index.js**, экспортирующий auth и listings



Добавляем authStore в App.vue:

```
1  <script setup>
2  | import { onMounted } from 'vue'
3  | import BaseLayout from '@layout/BaseLayout.vue'
4  | import { useAuthStore } from '@stores'
5  |
6  | onMounted(() => {
7  |   useAuthStore().initFromStorage()
8  | })
9  | </script>
10 |
11 | <template>
12 |   <BaseLayout />
13 | </template>
14 |
15 | <style scoped>
16 | </style>
17 |
```

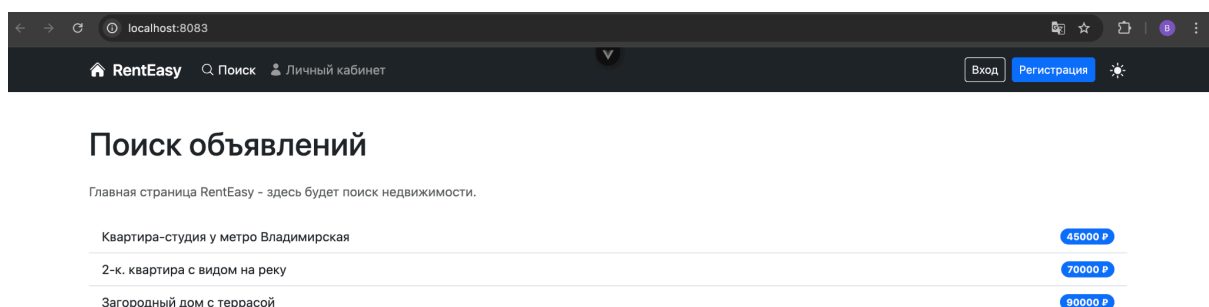
И listingsStore в HomeView:

```

1  <template>
2    <div class="container py-4">
3      <h1 class="mb-4">Поиск объявлений</h1>
4      <p v-if="!listingsStore.loading && !listingsStore.error" class="text-muted mb-4">
5        Главная страница RentEasy - здесь будет поиск недвижимости.
6      </p>
7
8      <div v-if="listingsStore.loading" class="text-muted">Загрузка объявлений...</div>
9      <div v-else-if="listingsStore.error" class="alert alert-danger">
10       {{ listingsStore.error }}
11     </div>
12     <ul v-else-if="listingsStore.hasListings" class="list-group list-group-flush">
13       <li
14         v-for="item in listingsStore.listings"
15         :key="item.id"
16         class="list-group-item d-flex justify-content-between align-items-center"
17       >
18         <span>{{ item.title }}</span>
19         <span class="badge bg-primary rounded-pill">{{ item.price }} ₽</span>
20       </li>
21     </ul>
22     <p v-else class="text-muted">Объявлений пока нет.</p>
23   </div>
24 </template>
25
26 <script setup>
27 import { onMounted } from 'vue'
28 import { useListingsStore } from '@stores'
29
30 defineOptions({ name: 'HomeView' })
31
32 const listingsStore = useListingsStore()
33
34 onMounted(() => {
35   listingsStore.fetchListings()
36 })
37 </script>

```

Результат:



## 5. Доделываем страницы и стили

- 1) На страницы входа и регистрации вместо заглушек добавлены формы.

```

<template>
  <div class="auth-wrapper d-flex align-items-center py-5">
    <div class="container">
      <div class="row justify-content-center">
        <div class="col-md-8 col-lg-6">
          <section class="card shadow-sm" aria-labelledby="login-title">
            <div class="card-body p-4">
              <h1 class="h4 mb-3" id="login-title">Вход</h1>
              <p class="text-muted small mb-4">
                Войдите в аккаунт, чтобы управлять объявлениями и бронированиями.
              </p>
              <form class="row g-3" @submit.prevent="handleSubmit">
                <div class="col-12">
                  <label for="loginEmail" class="form-label">Email</label>
                  <input
                    id="loginEmail"
                    v-model="form.email"
                    type="email"
                    class="form-control"
                    placeholder="name@example.com"
                    required
                  />
                </div>
                <div class="col-12">
                  <label for="loginPassword" class="form-label">Пароль</label>
                  <input
                    id="loginPassword"
                    v-model="form.password"
                    type="password"
                    class="form-control"
                    required
                  />
                </div>
                <div v-if="authStore.loginError" class="col-12">

```

- 2) На страницу поиска вместо заглушки добавлен баннер и форма с фильтрацией, карточка объявления вынесена в components.

```

1  <template>
2    <div class="card h-100 shadow-sm">
3      <div class="card-body">
4        <h3 class="card-title h6">{{ listing.title }}</h3>
5        <p class="card-text small text-muted mb-2">{{ listing.address }}</p>
6        <p class="fw-semibold mb-0">{{ formatPrice(listing.price) }} / месяц</p>
7        <div v-if="listing.rooms || listing.area" class="small text-muted mt-1">
8          {{ listing.rooms ? `${listing.rooms} к.` : '' }}
9          {{ listing.area ? `${listing.area} м²` : '' }}
10         {{ listing.type ? `· ${listing.type}` : '' }}
11        </div>
12      </div>
13    </div>
14  </template>
15
16  <script setup>
17    defineOptions({ name: 'ListingCard' })
18
19    defineProps({
20      listing: {
21        type: Object,
22        required: true,
23      },
24    })
25
26    function formatPrice(price) {
27      if (price == null) return '-'
28      return new Intl.NumberFormat('ru-RU').format(price) + ' ₽'
29    }
30  </script>
31

```

- 3) Добавлен профиль: если пользователь не залогинен, сообщение и предложение войти. Если залогинен, отображаются данные и объявления (также отображение входа на навбаре)

```
1 <template>
2 <div v-if="!authStore.isLoggedIn" class="container py-5 text-center">
3   <p class="text-muted mb-3">Вы не авторизованы.</p>
4   <router-link to="/login" class="btn btn-primary">Войти</router-link>
5 </div>
6
7 <div v-else>
8   <header class="profile-header py-4 mb-4" role="banner">
9     <div class="container">
10      <div class="row align-items-center g-3">
11        <div class="col-auto">
12          <div class="avatar-circle">{{ avatarInitials }}</div>
13        </div>
14        <div class="col">
15          <h1 class="h4 mb-1">{{ authStore.userName || 'Пользователь' }}</h1>
16          <p class="mb-1 small">{{ authStore.userEmail }}</p>
17          <span class="badge bg-light text-dark" style="font-size: 0.75rem">Арендатор и арендодатель</span>
18        </div>
19      </div>
20    </div>
21  </header>
22
23  <main class="container mb-5" role="main">
24    <div class="row g-4">
25      <section class="col-lg-4" aria-labelledby="profile-sidebar-title">
26        <div class="card shadow-sm mb-4">
27          <div class="card-body">
28            <h2 class="h5 mb-3" id="profile-sidebar-title">Профиль</h2>
29            <dl class="row mb-0 small">
30              <dt class="col-5 text-muted">Имя</dt>
31              <dd class="col-7">{{ authStore.userName || '-' }}</dd>
32              <dt class="col-5 text-muted">Email</dt>
33              <dd class="col-7">{{ authStore.userEmail || '-' }}</dd>
34              <dt class="col-5 text-muted">Телефон</dt>
35              <dd class="col-7">{{ authStore.user?.phone || '-' }}</dd>
36              <dt class="col-5 text-muted">Город</dt>
37              <dd class="col-7">Санкт-Петербург</dd>
38            </dl>
39          </div>
40        </div>
41      </section>
42    </div>
43  </main>
44 </div>
```

- 4) В assets добавлен theme.css со стилями тем (для будущего composable)

```
1 /* Светлая тема */
2 :root {
3   --body-bg: #ffff;
4   --body-bg-alt: #f5f5f7;
5   --text-primary: #212529;
6   --text-muted: #6c757d;
7   --hero-gradient: linear-gradient(120deg, #0d6efd 0%, #6f42c1 100%);
8   --hero-overlay: rgba(0, 0, 0, 0.25);
9   --profile-header-gradient: linear-gradient(135deg, #0d6efd 0%, #6610f2 100%);
10  --card-bg: #ffff;
11  --card-border: rgba(0, 0, 0, 0.125);
12  --footer-border: #dee2e6;
13  --navbar-bg: #212529;
14  --theme-toggle-bg: rgba(255, 255, 255, 0.15);
15  --theme-toggle-color: #ffff;
16  --theme-toggle-hover-bg: rgba(255, 255, 255, 0.25);
17 }
18
19 /* Тёмная тема */
20 [data-theme="dark"] {
21   --body-bg: #1a1a1a;
22   --body-bg-alt: #121212;
23   --text-primary: #e9ecef;
24   --text-muted: #adb5bd;
25   --hero-gradient: linear-gradient(120deg, #0a4a8f 0%, #4a1a6f 100%);
26   --hero-overlay: rgba(0, 0, 0, 0.45);
27   --profile-header-gradient: linear-gradient(135deg, #0a4a8f 0%, #3d0d99 100%);
28   --card-bg: #2d2d2d;
29   --card-border: rgba(255, 255, 255, 0.12);
30   --footer-border: #404040;
31   --navbar-bg: #0d1117;
32   --theme-toggle-bg: rgba(255, 255, 255, 0.15);
33   --theme-toggle-color: #e9ecef;
34   --theme-toggle-hover-bg: rgba(255, 255, 255, 0.25);
35 }
36
37 body {
```

## 6. Composable (смена темы) и картинки в карточках

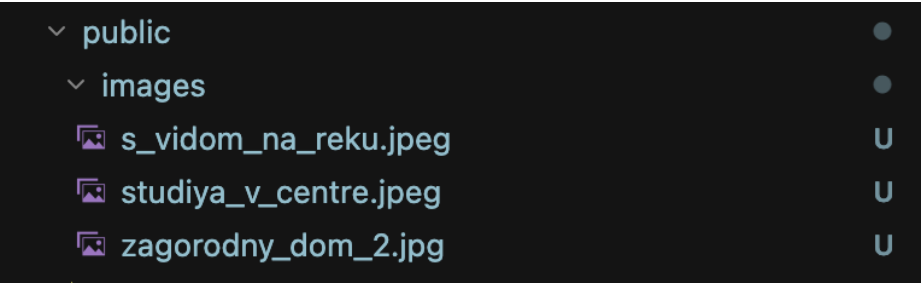
Я использую composable для смены темы и сохранения выбора.

Файл с логикой (useTheme.js):

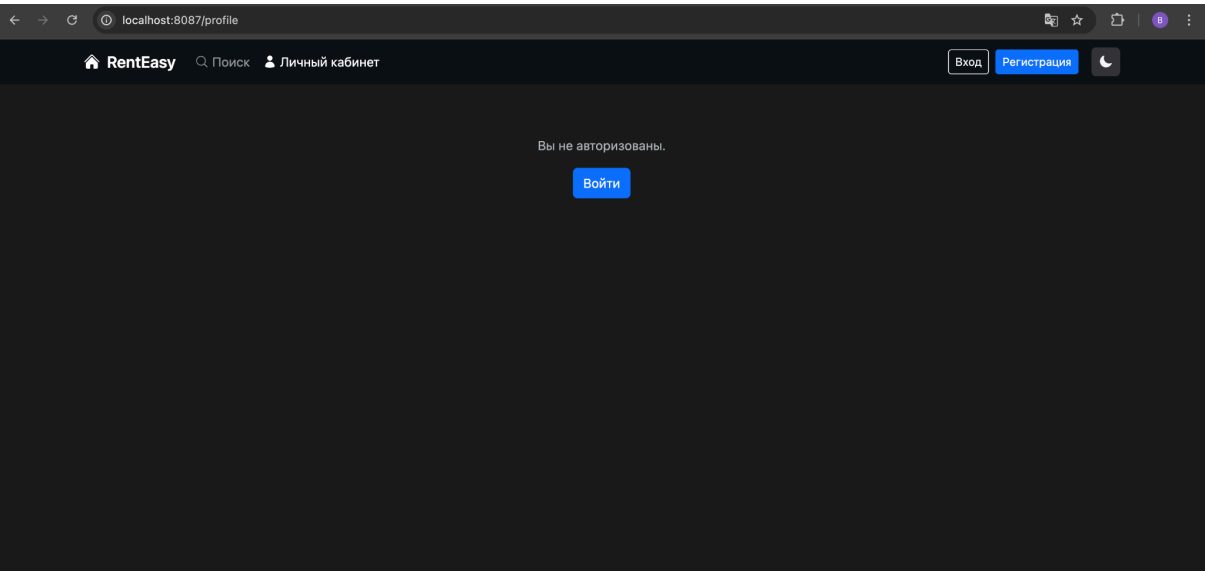
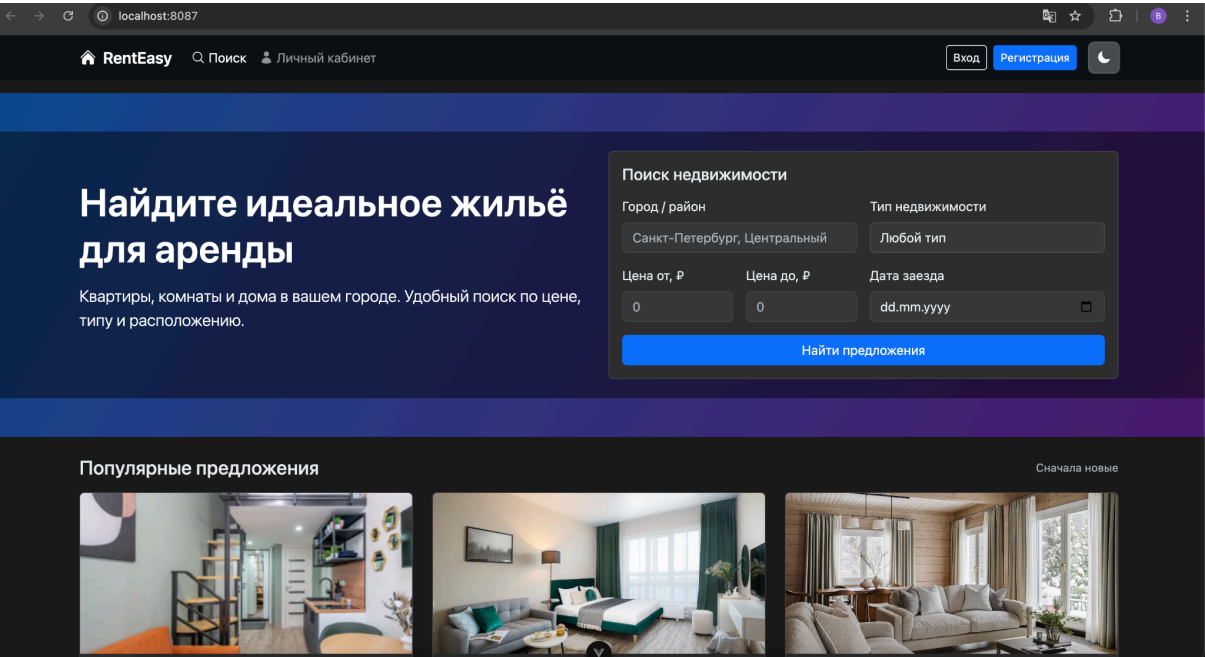
```
7 function getStored() {
8   try {
9     return localStorage.getItem(STORAGE_KEY)
10  } catch {
11    return null
12  }
13 }
14
15 function applyToDocument(theme) {
16   const html = document.documentElement
17   if (theme === DARK) {
18     html.setAttribute('data-theme', DARK)
19   } else {
20     html.removeAttribute('data-theme')
21   }
22 }
23
24 export function useTheme() {
25   const theme = ref(getStored() === DARK ? DARK : LIGHT)
26
27   const isDark = computed(() => theme.value === DARK)
28
29   const toggleLabel = computed(() =>
30     isDark.value ? 'Включить светлую тему' : 'Включить тёмную тему'
31   )
32
33   /** id иконки в спрайте: солнце или луна */
34   const iconId = computed(() => (isDark.value ? 'icon-moon' : 'icon-sun'))
35
36   function setTheme(value) {
37     if (value !== LIGHT && value !== DARK) return
38     theme.value = value
39     applyToDocument(value)
40     try {
41       localStorage.setItem(STORAGE_KEY, value)
42     } catch {}
43   }
44
45   function toggle() {
46     setTheme(theme.value === LIGHT ? DARK : LIGHT)
47   }
48 }
```

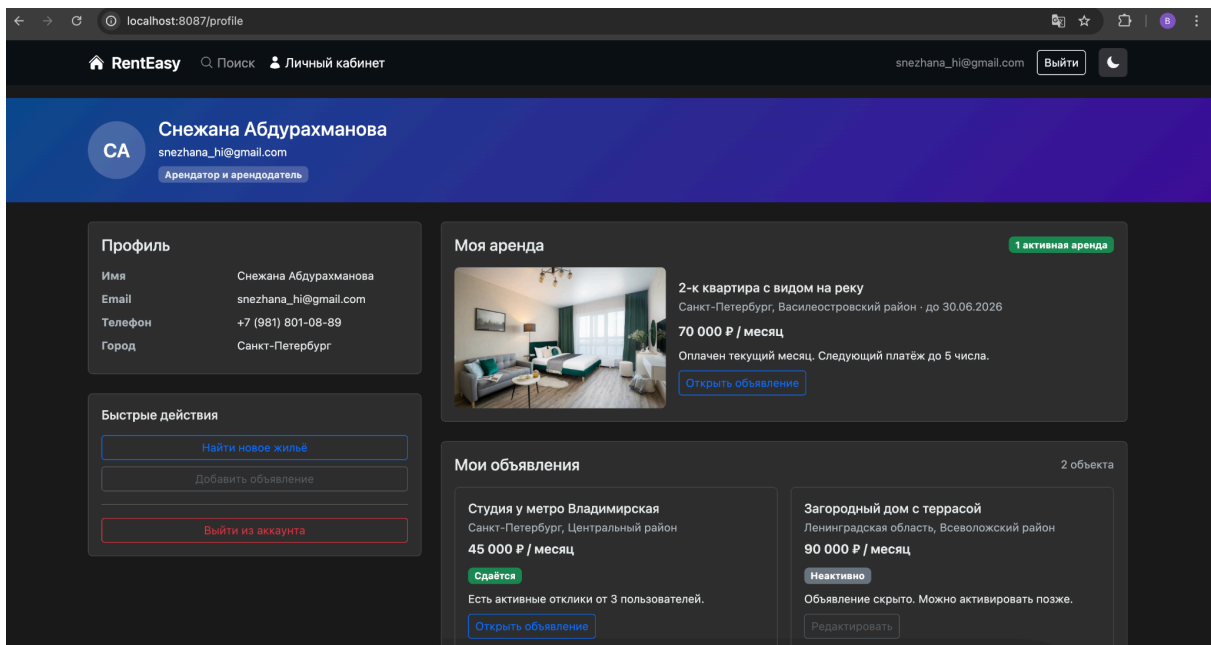
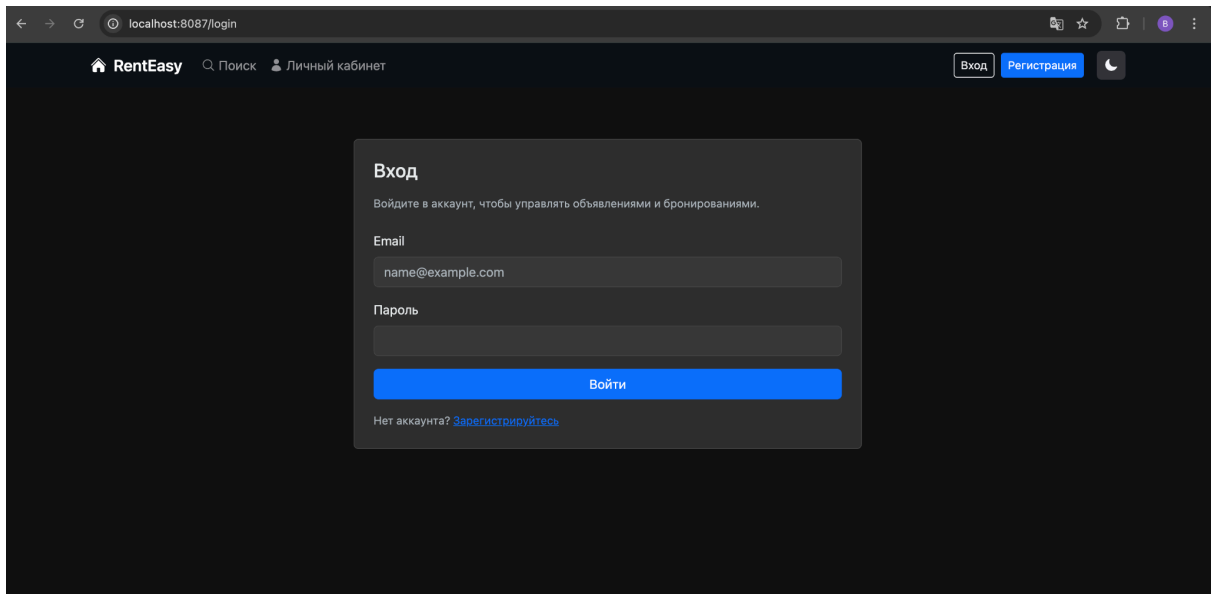
Composable хранит текущую тему в ref и в localStorage, при смене ставит или снимает data-theme="dark" на <html>, возвращает toggle, isDark, iconId и др. для кнопки переключения в навбаре.

Также добавлены картинки для карточек с объявлениями:



Итоговый результат:





## Вывод

В работе я более подробно углубилась во Vue.js и научилась работать с ним на практике и разрабатывать большой проект.